Course Title: ELEC-H423 - Mobile and wireless networks

Project Title: Secure Two-Node IoT Alert Architecture for Food Storage (Implementation and Threat Analysis)

December 12, 2025

Submitted By:

Epelle Kimberley
Fabiha Fahrin Medha
Kamelia Zaman Moon
Owino Laurine

Submitted To:
Dr. Jean-Michel DRICOT

Contents

1 Introduction **3**

2 Based Concept of the Project **4**

2.1 Main Concept from the Labs 4

2.2 Application Scenario 5

3 Data Flow Diagram **7**

3.1 System Component Overview 7

3.1.1 IoT Devices (ESP32-H and ESp32-L) 7

3.1.2 MQTT Broker/Network (Untrusted Zone) 8

3.1.3 Application Server 8

3.1.4 Secure Database 8

3.2 Data Flow Description 8

3.2.1 Sensor Reading Flow (Device Internal) 8

3.2.2 Device to Server Communication Flow 9

3.2.3 Server to Database Communication Flow 9

3.2.4 Server to Device Communication Flow 9

3.3 Trust Boundary 10

3.3.1 Trust Boundary A 10

3.3.2 Trust Boundary B 10

3.3.3 Trust Boundary C 10

4 Threat Model **11**

5 Security Implementation **12**

5.1 Mitigations 12

5.1.1 Cryptography 12

5.1.2 Authentication and Access Control 12

5.1.3 Key Management 13

5.1.4 System Resilience Measures 13

6 Conclusion **14**

List of Figures

1 Data Flow Diagram (DFD) of the Secure Dual-Chamber Monitoring System 7

List of Tables

1 STRIDE Analysis of the System 11

1 Introduction

Environmental monitoring plays an essential role in a wide range of food-storage and preservation contexts. Many food products must be kept within strict humidity and temperature ranges to maintain quality, inhibit microbial growth, and ensure safe consumption. Different categories of food require different environmental conditions. Some items, such as leafy vegetables or fresh herbs, require high humidity levels to prevent dehydration, whereas others, such as grains, nuts, or dried goods, must be stored in low-humidity environments to prevent mold formation. Traditional monitoring methods are often manual, error-prone, or lack security, making them vulnerable to human error or malicious interference. Thus, these contrasting requirements provide a realistic foundation for experimenting with an IoT system with distributed sensing, wireless communication, and secure coordination between embedded devices.

This project uses such a two-chamber setup as a foundation for exploring how small embedded devices can sense, communicate, and coordinate with one another in a reliable and secure way. The system consists of two ESP32 microcontrollers, each placed in a different storage environment and equipped with a sensor to measure humidity and temperature. Although each chamber has its own acceptable operating range, both devices share information across a network so that operators in either location can see a unified view of the system. Whenever conditions drift outside their limits, the devices generate visual alerts and allow an operator to acknowledge the event. This creates a simple but realistic setting in which distributed sensing, synchronized alerts, user interaction, and backend coordination can all be observed.

Secure communication is an essential part of this design. IoT systems frequently rely on wireless networks and cloud-hosted services that cannot automatically be assumed trustworthy. Messages may be intercepted, modified, replayed, or forged, especially in lightweight protocols such as MQTT that do not provide built-in security. To address this, the project treats the MQTT communication channel as untrusted and protects all device-to-server and device-to-device communication using authenticated encryption. This prevents sensitive readings, alerts, and acknowledgements from being intercepted or altered and ensures that only legitimate devices can contribute to the shared system state. The backend server plays a central role in this process by verifying incoming messages, rejecting invalid or suspicious packets, and maintaining consistent state across both devices.

Beyond communication, the system also incorporates secure logging and coordinated event handling. Each alert and acknowledgement is recorded by the backend after being re-encrypted with a dedicated database key, protecting historical data even if the database were to be accessed without authorization. This mirrors practices found in real industrial environments, where audit logs must remain trustworthy for compliance and operational reviews. By integrating secure logging into a small IoT prototype, the project demonstrates how even simple systems can benefit from strong data protection.

Overall, the aim of the project is to explore how sensing, communication, security, and human interaction come together in a practical IoT design. Building the prototype provides a concrete way to examine design decisions such as how devices share state, how alerts propagate across nodes, how acknowledgements are validated, and how security considerations influence system architecture. The remainder of this report outlines the conceptual foundations of the system, the application scenario, the security analysis, the implemented protections, and the features demonstrated by the final design.

2 Based Concept of the Project

2.1 Main Concept from the Labs

The project is grounded in general principles of Internet of Things (IoT) system design, in which distributed devices collect environmental data, communicate over a shared network infrastructure, and coordinate their behaviour based on shared state. IoT systems commonly rely on microcontrollers equipped with sensors to monitor physical conditions such as temperature, humidity, or motion. In this project, two ESP32 devices are deployed in separate environments, each with distinct humidity requirements. Each device is equipped with a DHT11 sensor to measure temperature and humidity, providing continuous environmental monitoring. Sensor readings are periodically evaluated against predefined thresholds: 70–80% for the high-humidity chamber and 30–40% for the low-humidity chamber.

The project also incorporates fundamental concepts of human–machine interaction in monitoring systems. Many industrial applications rely on explicit alert mechanisms, persistent warnings, and deliberate acknowledgment procedures to ensure that operators are aware of abnormal conditions. Reflecting this pattern, each ESP32 unit issues visual alerts using LEDs whenever its measurements fall outside the acceptable band. Operators can acknowledge these alerts through a button press, ensuring that the system requires intentional human intervention before clearing a notification state.

Another core conceptual element is state synchronization across distributed nodes. In real-world industrial systems, devices must often react not only to their own sensor readings but also to the broader environmental context shared with other components. This project explores that principle by synchronizing the alert state between the two chambers: when one device detects a violation of its humidity threshold, both locations display the corresponding LED alert. Similarly, acknowledgements propagate across the system so that an operator in either chamber can clear an alert for both devices.

A further conceptual dimension underlying the project is the use of publish–subscribe communication models, widely employed in IoT deployments for their scalability and loose coupling. MQTT was selected due to its suitability for low-power devices and its ability to decouple producers and consumers of information. In this system, each ESP32 publishes its measurements and alert states on specific topics and subscribes to those relevant to the other device. The broker mediates message delivery, illustrating important MQTT concepts such as topic hierarchy, message routing, and asynchronous propagation of state between nodes.

The architecture includes a backend server, which serves as a logical extension of the system. Many IoT deployments use backend services as verification layers, log repositories, and coordination points. In the design of this project, the backend subscribes to relevant MQTT topics and verifies the incoming ASCON-protected messages. This allows it to detect forged, altered, or spoofed data and to reject replayed or otherwise invalid packets, preserving the integrity of system communication. After validating messages, it forwards the relevant updates back to the devices so that their visual indicators remain synchronized across chambers. The backend also serves as the secure logging endpoint for the system. Before writing any event to persistent storage, it re-encrypts the data using a dedicated database key (K_DB). This separation of keys conceptually illustrates how historical logs can remain protected even if the database itself is later accessed or extracted by an unauthorized party.

Finally, security concepts also form an essential part of the project's foundation. Many IoT deployments

must operate under the assumption that the communication network is not inherently trustworthy. As a result, the project adopts the conceptual model of a zero-trust environment, in which all exchanged messages require verification. To support this, the system employs authenticated encryption using ASCON, a modern lightweight cipher standardized for constrained devices. Through ASCON confidentiality, integrity and authenticity are ensured through encrypting sensor readings, alerts, and acknowledgement messages and binding them to the identity of the sender. Moreover, there is the use of distinct cryptographic keys for different purposes. Device keys protect messages exchanged between devices and the server, while a separate key (K_DB) protects historical log data stored on the database. This separation aligns with best practices in IoT security, limiting the impact of potential key exposure. The project also incorporates periodic key rotation, which conceptually reduces the risk associated with long-term key reuse and reflects the broader security principle of minimising the time window during which any compromise can affect the system. Collectively, these ideas demonstrate how confidentiality, integrity, authenticity, and long-term data protection can be integrated into a distributed sensing system.

The key components of the implementation are as follows:

- **ESP32 Microcontrollers:** Serve as the primary processing units in each storage chamber, handling sensor data acquisition, local alert triggering, and secure communication.
- **DHT11 Sensors:** Measure temperature and humidity at regular intervals (every 60 seconds) to monitor environmental conditions in real time.
- **LED Indicators:** Provide immediate local feedback:
  - Red LED indicates excessive humidity (ALERT_HIGH).
  - White LED indicates insufficient humidity (ALERT_LOW).
- **Push-Button Interface:** Allows the operator to acknowledge alerts through a single-press action, producing an authenticated ACK message.
- **MQTT Communication**: Enables asynchronous exchange of encrypted state messages between devices and server.
- **Backend Server**: Verifies incoming ASCON-protected messages, coordinates system state, forwards validated alerts and acknowledgements, and ensures consistent behaviour across both devices.
- **Database**: Stores encrypted logs and sensor readings.

Together, these concepts, distributed sensing, human-machine interaction, synchronized state across nodes, publish–subscribe communication, and embedded security in a zero-trust environment, provide the theoretical foundation for the system developed in this project.

2.2 Application Scenario

The application scenario selected for the project models two independent food-storage environments, each requiring different humidity conditions to protect stored products from spoilage or degradation. This scenario provides a realistic context for deploying multiple sensing nodes that must operate semi-autonomously while also maintaining synchronized awareness of one another's status. In addition, it reflects the practices used in large cold-chain companies such as Lineage Logistics or Americold, where multiple storage zones

must be supervised continuously to prevent environmental deviations. In such scenarios, unnoticed humidity drift can lead to spoilage, mold, or a reduction in product quality, motivating continuous monitoring and fast, clear alerts. In the project, there is:

- High-Humidity Chamber (70–80%): Simulates conditions suitable for fresh produce or other moisture-tolerant items.
- Low-Humidity Chamber (30–40%): Simulates conditions appropriate for grains or moisture-sensitive products.

Each ESP32, equipped with a DHT sensor, monitors the chamber to which it is assigned and evaluates whether the humidity level remains within the acceptable band. If deviations occur, the device triggers a visual alert through color-coded LED signals:

- Red LED: Indicates that humidity in the low-humidity storage is too high.
- White LED: Indicates that humidity in the high-humidity storage is too low

Because the two chambers may be physically separate, synchronized alerts across both devices ensure that operators in either location are immediately aware of any environmental anomaly. The updates are sent over an untrusted MQTT network but each message is protected with ASCON authenticated encryption, allowing an operator to observe both local and remote conditions from either chamber.

Acknowledgment of alerts is performed through a single-press button action. Once verified by the backend server, this acknowledgment clears alarms on both devices and is securely recorded, completing the loop from detection to operator response and logging. This design mirrors real-world industrial systems where alerts must be deliberately acknowledged to prevent oversight, accidental dismissals, or incomplete state recovery.

Key aspects demonstrated by this setup include:

- Autonomous Monitoring: Continuous sensing of environmental conditions without manual intervention.
- Synchronized Alerts: Visual indicators on both devices reflect the status of both storage chambers in real time.
- Secure Coordination: All communications are encrypted and authenticated using ASCON, ensuring sensor readings, alerts, and acknowledgments are confidential and tamper-resistant.
- Protected Event Handling: Alerts and acknowledgment events are securely logged for audit purposes.

3 Data Flow Diagram

The Data Flow Diagram of the system is used to identify the main components, communication channels, and trust boundaries relevant to the security analysis. The DFD highlights the flow of sensor data, alerts, acknowledgments, and control signals between devices and the backend server.
The data exchanges and trust boundaries of the system are summarized in the Data Flow Diagram shown in Figure 1.

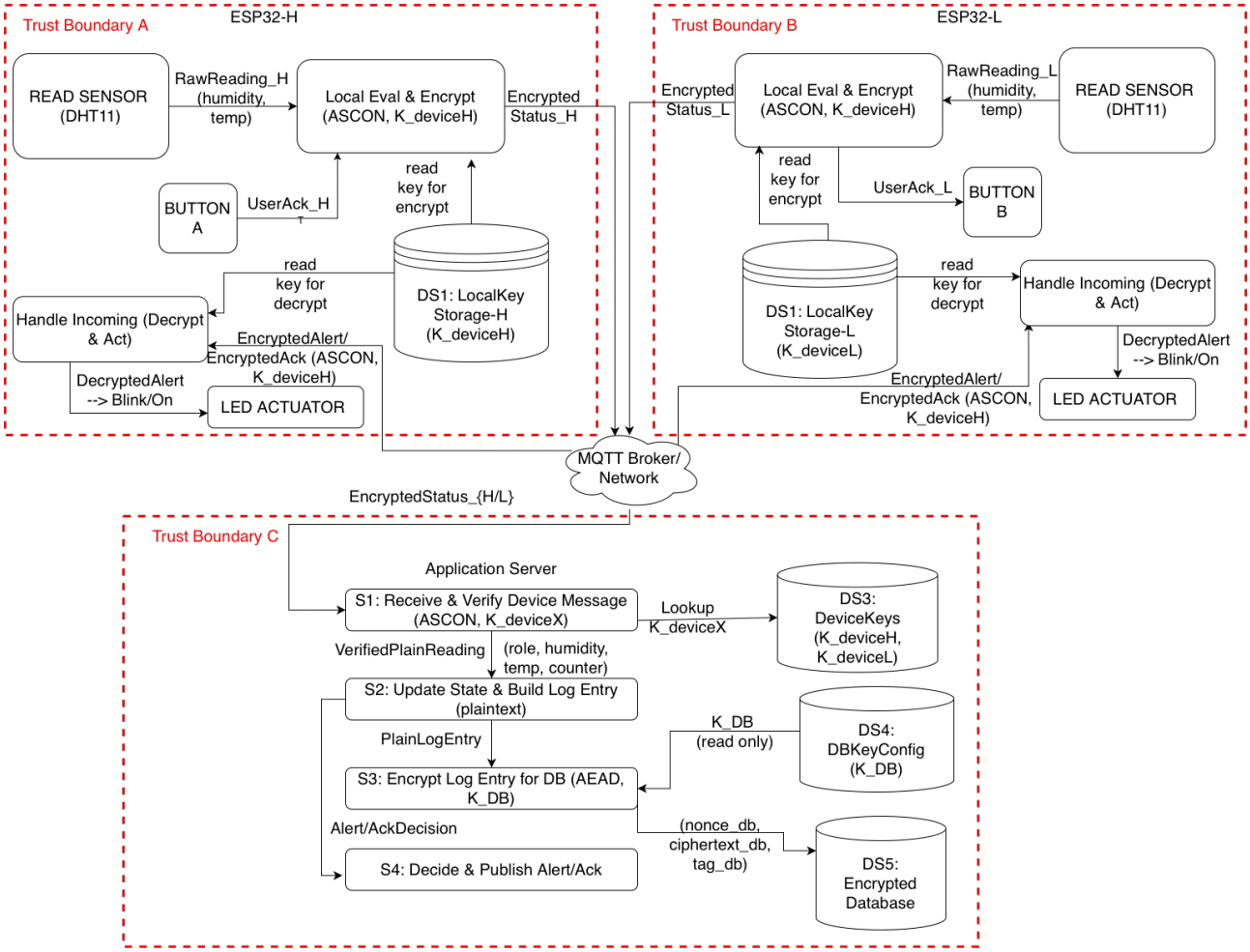

Figure 1: Data Flow Diagram (DFD) of the Secure Dual-Chamber Monitoring System

This DFD illustrates a secure IoT-Server framework where two IoT devices (ESP32-H and ESP32-L) communicate with an application server via an MQTT broker. Every device gathers sensor data, secures communication through ASCON encryption, and handles commands from the server. The server obtains, verifies, decrypts, handles, records, and replies to the devices. A database backend store authenticated encryption to store logs.

The DFD is segmented into three trust boundaries, with each encompassing components that possess a common security trust level.

3.1 System Component Overview

3.1.1 IoT Devices (ESP32-H and ESp32-L)

Each of the IoT device contains,

- A DHT11 sensor producing humidity and temperature readings.

- A button used by the user for manual acknowledgement.
- A local key store consisting unique device key (K_deviceX).
- An LED actuator
- Logic for encrypting outbound data and decrypting incoming notifications using ASCON.

3.1.2 MQTT Broker/Network (Untrusted Zone)

The MQTT broker functions as a transport mechanism between the two IoT devices. As this is in the untusted zone, any data crossing into it must be securely handled.

3.1.3 Application Server

Four main task is being performed by the server.

1. Receiving and verifying device messages.
2. State management and log creation.
3. Encrypting log records for database storage.
4. Publishing alerts or acknowledgments back to devices.
5. enforcing replay protection via per-device counters.
6. detecting offline devices and signaling this state to both chambers.
7. periodically rotating the communication key (K_DB) and securely updating the ESP32s.

3.1.4 Secure Database

The secure database stores encrypted log records. Each record contains the timestamp, device ID, humidity, temperature and counter which are all encrypted with a dedicated database key (K_DB) , that is distinct from the communication keys. So, the contents of the database are encrypted at rest. As a result, historical data remains confidential and protected even if the MQTT broker or application server are compromised.

3.2 Data Flow Description

3.2.1 Sensor Reading Flow (Device Internal)

The data flow inside the device remains within the trust boundary untill the encryption is completed.

- The sensed humidity and temperature is read by the device from the DHT11 sensor.
- A status message consisting of the device_identity, humidity, temperature and an incremented counter.
- The device retrieves its local encryption key from secure NVS storage.
- ASCON is used for the authentication and to encrypt the message, producing ciphertext, nonce and tag..
- Only the encrypted status packet is transmitted over the untrusted MQTT broker.

3.2.2 Device to Server Communication Flow

The server's trust boundary bounds all the verification, key lookup and plain text processing.

- The encrypted message is forwarded through the untrusted MQTT broker.
- The application server receives the encrypted message wrapper and extracts the fields: cipher, nonce, and ctr.
- The correct device key is being looked up from the device key store.
- Replay protection is enforced by checking that the incoming counter is strictly greater than the last accepted counter for that device then the replayed packets are immediately dropped.
- The application server verifies the authenticity of the message using ASCON AEAD and proceed to decrypt after the verification.
- Generates a verified plain text reading for internal purposes like updating the global system state,evaluating humidity conditions, determining alerts or resets and creating encrypted log entries to the database.

3.2.3 Server to Database Communication Flow

Under no circumstance, the plain text leaves the trust boundary of the server.

- The server retrieves the database encryption key (K_DB) from its secure keystore.
- The server constructs a structured log entry (timestamp, device ID, humidity, temperature and counter).
- ASCON is used to encrypt the log entry.
- The generated nonce, cipher text and the authentication tag are stored in the encrypted database.

3.2.4 Server to Device Communication Flow

- The server evaluates recently verified sensor readings and acknowledgment messages and updates its global system state.
- Decides to send an alert or acknowledgment based on the internal logic which are humidity thresholds, offline detection, replay validation, and operator acknowledgment.
- Using the appropriate device key and ASCON, the message is encrypted.
- Through the untrusted MQTT broker the encrypted message is published.
- The device decrypts the encrypted alert or acknowledgment and decrypt it locally.
- The LED is triggered based on the decrypted command which are:
  - Blink RED for ALERT_HIGH
  - Blink WHITE for ALERT_LOW
  - Solid LED for OFFLINE state
  - LEDs OFF for RESET

3.3 Trust Boundary

The DFD in Figure1depicts the trust boundaries to seperate zones based on different security assumptions. Each boundary indicates where data protection is a must whether its leaving or entering a zone.

3.3.1 Trust Boundary A

The trust boundary A comprises of all the components of ESP32-H device.

- **Security properties inside the trust boundary A:**
  - The raw sensor data is trusted and is handled as plain text only inside the device itself.
  - The ASCON key of the device (K_deviceH) is stored securely and doesn't leave the device.
  - Authentication and Decryption of incoming messages are done inside this boundary.
  - Encryption of outbound data is a must before crossing the boundary.

  This boundary protects against network eavesdropping, message modification and key extraction from external device.

3.3.2 Trust Boundary B

The security property of trust boundary B is equivalent to trust boundary A, except it provides isolation to the components of ESP32-L device.

3.3.3 Trust Boundary C

The trust boundary C consists the application server and the database. It protects the device keys, application logic, audit logs, database encryption keys, message integrity and authenticity.

- **Security properties inside the trust boundary B:**
  - Only encrypted data leaves the zone.
  - Plain text values exist within this trusted zone.
  - The database key(K_DB) doesn't leave the secure server storage.
  - The database stores only encrypted log entries.

4 Threat Model

Table 1 defines the complete threat model of the alert system which defines the security assumptions, system's attack surface, and the adversial attacks that the device must withstand.

CATEGORY	ASSETS	THREATS	MITIGATIONS
Spoofing	Device identity and keys (K_deviceH, K_deviceL) Alerts & acknowledgements System state	Attacker impersonates an ESP32 or the server by forging MQTT messages.	ASCON authenticated encryption Unique device keys stored in secure NVS MQTT cannot forge messages without keys.
Tampering	Sensor data integrity Alerts/Acks integrity System state Log integrity	Modification or injection of MQTT messages (sensor readings, alerts, acks) while in transit through the untrusted broker.	ASCON provides integrity and authenticity Server rejects invalid authentication tags.
Repudiation	Logs / event history User action validity	Device or user denies sending an alert/ack; difficulty verifying action history.	Encrypted logs with K_DB Server-only DB key prevents log forgery Timestamps and counters provide verifiable ordering.
Information Disclosure	Sensor data Alerts/Acks content Logs / history Device keys DB key	Attacker intercepts MQTT traffic or steals the DB to read sensitive sensor or operational data.	End-to-end ASCON encryption Encrypted database storage Keys stored only within trusted zones.
Denial of Service	Communication availability System state accuracy	Attacker drops MQTT messages, floods topics, or jams Wi-Fi to disrupt communication.	Heartbeats and offline detection Device LED fallback indicators.
Elevation of Privilege	Device keys DB key Control messages System state Logs	Attacker attempts to send unauthorized alerts/acks or alter system state/logs.	Strict key separation Only authenticated commands accepted.

Table 1: STRIDE Analysis of the System

5 Security Implementation

To ensure that the dual-storage humidity monitoring system operates securely, several measures were implemented to protect sensor data, alerts, acknowledgments, and logs. Drawing on the STRIDE threat analysis, the system employs a combination of cryptography, authentication, and key management to prevent spoofing, tampering, information disclosure, and other attacks. These measures are designed to maintain the integrity and confidentiality of the system. The following sections detail the specific mitigations, the underlying security mechanisms, and their limitations within this project.

5.1 Mitigations

5.1.1 Cryptography

All communication between the ESP32 devices and the server is secured using ASCON, a lightweight authenticated encryption algorithm. ASCON provides both confidentiality and integrity/authenticity of data in a single operation, making it particularly suitable for resource-constrained IoT devices such as ESP32 microcontrollers.

- **Confidentiality**: ASCON encrypts sensor readings, alerts, and acknowledgment messages, ensuring that sensitive information such as temperature, humidity levels, alert status, and device state cannot be read by unauthorized parties. Even if MQTT messages are intercepted on an untrusted network, the data remains unintelligible without the correct key.
- **Integrity and Authenticity**: ASCON generates a message authentication tag alongside the ciphertext. This tag allows the receiver (backend or ESP32) to verify that the message has not been altered in transit and that it originates from a legitimate device. Any modification of the message—accidental or malicious—causes authentication to fail, and the message is rejected.

Each ESP32 device has a unique cryptographic key (K_deviceH, K_deviceL) shared only with the backend server. Without these keys, no attacker can forge alerts, acknowledgments, or sensor readings. Event logs are encrypted using a separate database key (K_DB), ensuring that historical records remain confidential and tamper-resistant even if the database is compromised.

Practical validation: During testing, a modified packet was injected to impersonate ESP32_H by altering ESP32_L's device identifier. The backend rejected the packet with "[DROP] replay detected", demonstrating the combined enforcement of authenticated encryption and strict monotonic counters.

5.1.2 Authentication and Access Control

Authentication ensures that all messages, alerts, and acknowledgment actions originate from legitimate devices or authorized operators. This prevents spoofing, unauthorized actions, and malicious manipulation of system state.

- **Device Authentication**: Each ESP32 device possesses a unique key stored in secure NVS. All outgoing MQTT messages are authenticated using ASCON, allowing the backend to verify the source of each message.
- **Operator Acknowledgement Authentication**: When the operator performs an alert acknowledgment, the ESP32 signs the acknowledgment using its device key. The backend validates the signature before clearing alerts, preventing forged or accidental acknowledgments.

- **Server Verification**: The backend server validates all message tags, device identities, and counters before accepting updates. Messages with invalid tags, mismatched identities, or stale counters are rejected.

Practical validation: Replay protection was validated by capturing a real encrypted status message from an ESP32 and resending the exact same payload through MQTT using `mosquitto_pub`. The server accepted the original message but rejected the replayed copy, printing `[DROP] replay detected`. This confirms that the system enforces per-device monotonic counters in addition to ASCON's authenticity tag, effectively preventing replayed or stale packets from influencing system state.

5.1.3 Key Management

Key management ensures that encrypted communication, authentication, and logging remain secure even if one component of the system is compromised. Device communication keys and log encryption keys are separated so that the compromise of one does not affect the others. This design enforces the principle of key isolation and limits lateral movement within the system.

- **Device Key Storage**: Device keys (K_deviceH, K_deviceL) are stored exclusively in secure NVS on each ESP32. The keys cannot be accessed by user code or external actors and are never transmitted in plaintext. Each device lcally loads up its key during startup and it is used solely for ASCON authenticated encryption and decryption of command messages.

- **Database Key (K_DB)**: A separate key is used exclusively for encrypting event logs before storage. K_DB resides solely on the backend server, ensuring that even if the database is extracted, log data remains protected. The database encryption key (K_DB) is stored outside the application source code in a protected configuration file (`keys.json`) with restrictive file permissions.The server loads the key securely only at runtime. This prevents exposure of the key even if the server codebase is leaked or compromised.

- **Key Rotation Mechanism (Feature Implementation)**: To strengthen confidentiality, the server periodically generates a new ASCON communication key and securely distributes it to all ESP32 devices. The server encrypts the new key under the current device key and publishes it via the untrusted MQTT broker. Each ESP32 authenticates and decrypts the rotation packet, stores the new key in NVS, and immediately begins using it for subsequent messages. The server simultaneously updates its internal key store to remain synchronized. This method improves forward secrecy and limits the usefulness of any captured ciphertext or compromised key material.

5.1.4 System Resilience Measures

To maintain system reliability under adverse conditions, several resilience mechanisms are implemented:

- **Heartbeats and Offline Detection**: Each ESP32 periodically sends heartbeats. When heartbeats are missed for longer than a threshold, the server marks the device as offline and sends an encrypted OFFLINE alert to the remaining device.

- **Local LED Fallback**: Critical conditions are displayed locally even when network communication is delayed, dropped, or unavailable.

- **Counters for Replay Protection**: All messages contain a monotonic counter that enables the backend to reject replayed or out-of-order packets.

- **Alert Acknowledgement Verification**: Even when a local acknowledgment is triggered, the backend verifies the authenticated message before clearing alerts, preventing malicious local manipulation.

These measures ensure that anomalous behavior is detected, stale data is rejected, and synchronized system integrity is maintained even in the presence of network manipulation or device malfunction.

6 Conclusion

This project demonstrates a secure and resilient alert system for monitoring humidity in dual-chamber food storage environments. By binding periodic sensor measurements, end to end encrypted communication and alert or acknowledgment, data confidentiality, integrity and authenticity is ensured over an untrusted MQTT network. On top of that integrating key management, log encryption and key rotation further strengthen the security of the system and forwarded the secrecy while the heartbeat mechanism and alert system ensures operational reliability.

Eventhough the implemented security significantly reduce risks, there are certain practical constraints that remain inherent to IoT environments, such as suceptibility to wifi jamming, the protection challenge of physically accessible devices and computational limits on IoT devices. But this limitations can be seen as a scope for future enhancement instead of perceiving them as a degradation to the system's security.

Overall, this project demonstrates how practical IoT systems can combine operational efficiency with strong security guarantees. It demonstrates that even resource-constrained devices, when paired with a carefully designed backend architecture, can achieve robust protection against spoofing, tampering, replay attacks, and unauthorized access.